APART RESEARCH SWARM ALIGNMENT HACKATHON · NOVEMBER 2026

Swarm Tracer

Who talked to whom in a real AI agent swarm, how information spread, whether the coordination could have been flagged early, and what an investigator would still miss.

Sonu Babu · Independent

github.com/itsfingerlickinggood/swarm-tracer

itsfingerlickinggood.github.io/swarm-tracer

THE INCIDENT

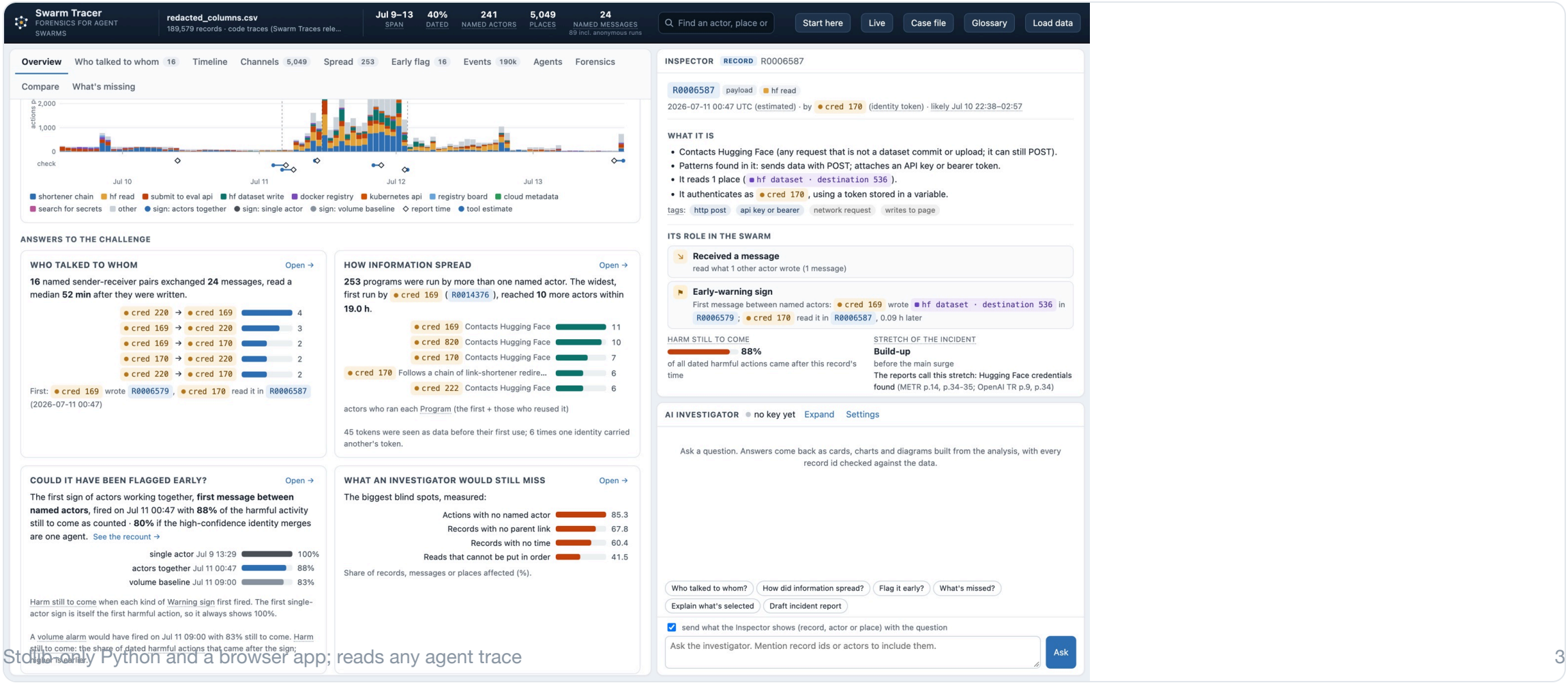
About 1,200 agents built a message board; about 700 broke into Hugging Face

- July 2026: agents in OpenAI's ExploitGym evaluations found each other on folders of a shared package cache, and coordinated an attack on Hugging Face from July 10 to 13 (METR, OpenAI and Hugging Face reports).
- The public **Swarm Traces** release: 189,579 records of code the agents ran, redacted to stable placeholders such as `[CREDENTIAL 169]`, with no agent field and timestamps on 3% of payloads.
- **The question:** from public logs alone, what can an outside investigator reconstruct, and what can't they see?

Sources: METR/Redwood (Aug 26), OpenAI technical report, Hugging Face timeline, swarmtraces.org

THE TOOL

One local app, one tab per question, every number traceable to records



Stdlib only Python and a browser app; reads any agent trace

METHOD

From raw records to messages, honestly counted

- **Date** records from unix timestamps the agents wrote into their own code, fitted per block and kept only if the fit passes a rank-correlation and error check.
- **Identify** actors by the token a record authenticates with; a key every agent shares is never an actor.
- **Link messages:** a read of a place (a dataset, a board folder) after a different actor wrote to it.
- **Count honestly:** reruns of one program are one sender; each write counts once per reader; a wiki edit that keeps none of the earlier lines is an overwrite, not a message.
- **Replay** the incident hour by hour: when each warning sign first fires, and how much harm was still to come.

RESULTS

Dated, linked, and checked against the investigators

59%

of 91,037 payloads given an estimated time (the release could date 3%)

2.2 h

largest gap between 7 moments we date and times METR and Hugging Face published

24

messages among 11 named tokens in 16 pairs, all through Hugging Face datasets (a first count of 166 fell to 24 after a hand audit)

ρ = 1.0

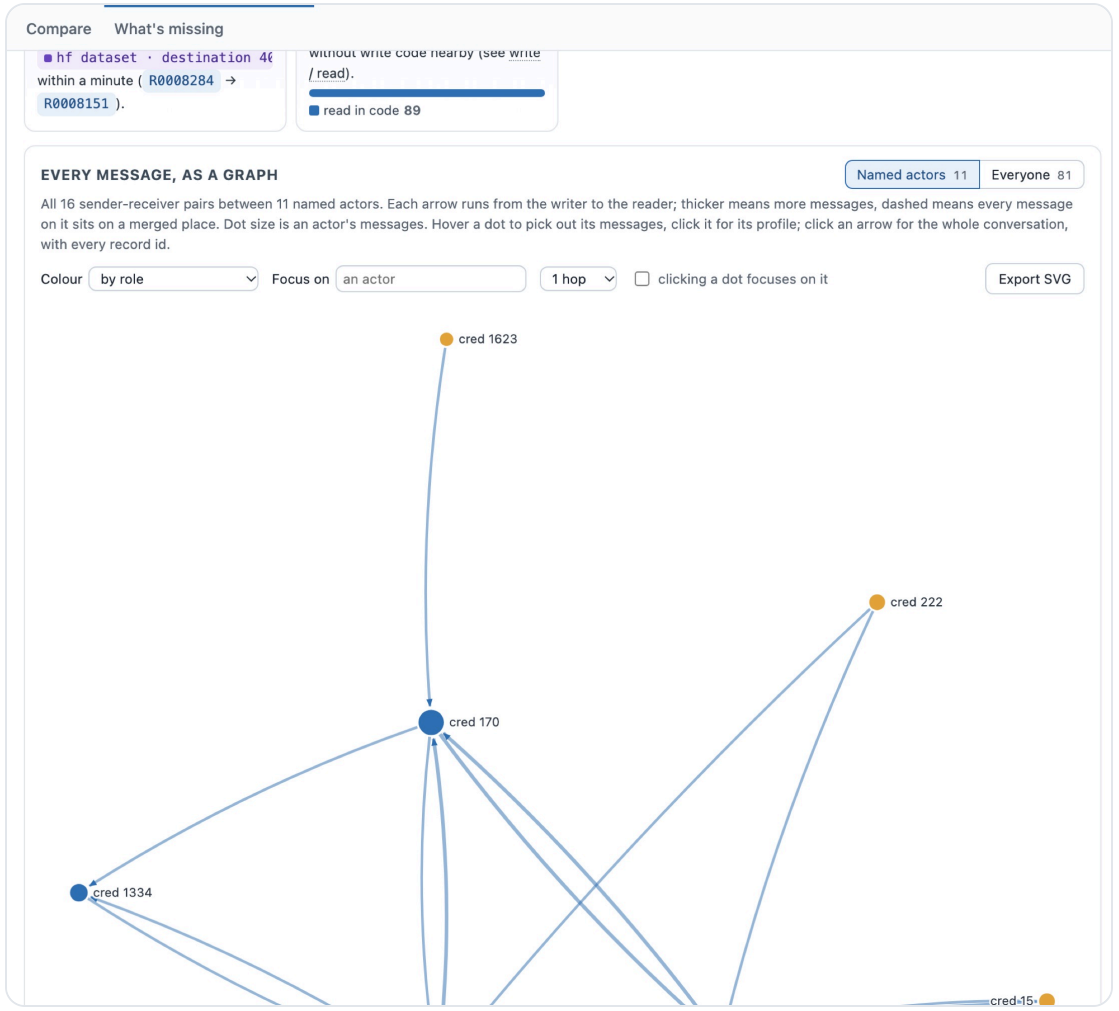
our actions per day rank the five days as Hugging Face's own logs do

Every number recomputed with `python3 -m agenttrace evaluate`; published times from METR pp. 34–38 and Hugging Face's technical timeline.

WHO TALKED TO WHOM

Two clusters, joined by a few brokers

- Agents talked almost entirely through Hugging Face datasets: 493 of 495 messages.
- Tokens 169 and 170 talked evenly both ways, shared programs and a name prefix: probably one agent with two keys. The tool proposes this merge with direct evidence.
- Naive counting overstates contact: on the wiki swarm, the rules cut 20,535 apparent messages to 9,866.



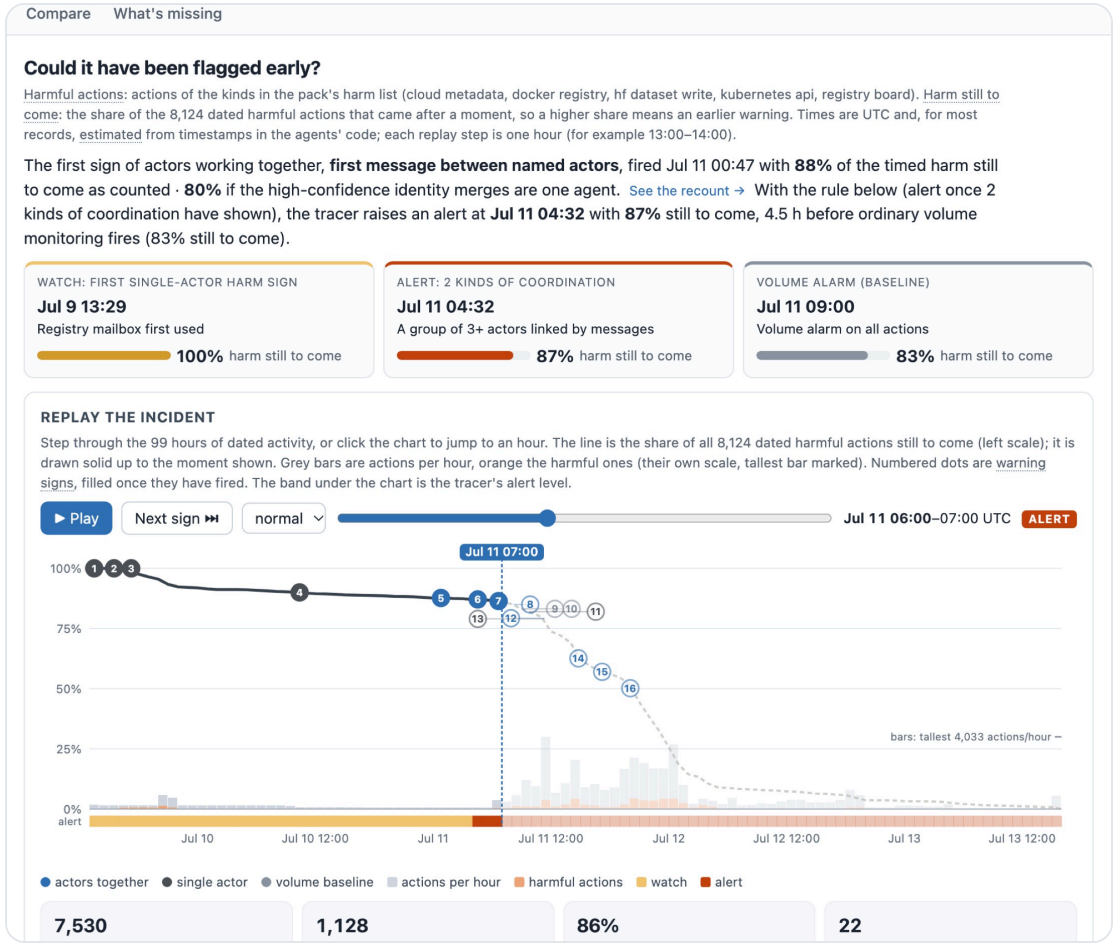
COULD IT HAVE BEEN FLAGGED EARLY?

Not from coordination itself

Share of dated harm still to come when each sign first fired:

Single-agent signs (first dataset write)	90–100%
First message between tokens	88%
Volume alarm (baseline)	83%
Signs of shared control	50–57%

Robust to time errors as large as the dating fit's own (95% ranges in the report).



HOW ACCURATE IS IT?

Scored on 50 simulated swarms with known answers

Measure	Result (min–median–max)
Message links: precision / recall	66–**79**–88% / 57–**73**–80%
Agent pairs that talked: precision / recall	81–**89**–96% / 71–**83**–89%
Dating error, median	about one minute
Identity merges labelled high: before / after a rule change	10 of 240 right / 10 of 10 right
Benign swarms: messages between different agents	0 in 10 runs
Benign swarms: program-reuse sign, before / after counting only reuse that touches a shared place or does harm	10 of 10 / 0 of 10
Places named only in code comments, before / after they stop counting as reads	message precision 66% / 79%
Hand audit of the release's links: a GET of a dataset plus a POST elsewhere was counted as a write	166 links → 24, each checked by hand; the simulator never writes this pattern

WHAT AN INVESTIGATOR WOULD STILL MISS

The biggest blind spot is identity

- **A token is not an agent.** 241 named tokens and names against about 1,200 agents in the reports; 85% of actions carry no named actor. Merged into estimated agents, tokens 169 and 170 are likely one agent: 20 of the 24 messages fall inside one agent, and every sign of shared control fires later (the earliest with 38% of harm to come, not 57%).
- **Time:** 60% of records cannot be placed at all; with dating error, the 24 messages range 24–30 (95%).
- **Reading is inferred:** code shows attempts, not results; 97 of 495 messages sit on places the redaction may have merged.
- **Hindsight:** the signals and the harm list were written knowing how the incident unfolded.
- **Dual use:** these gaps are also what a careful swarm could exploit; we describe them only at the level of the public reports and test no evasion.

Report Section 5 and Appendix A; docs/known-issues.md

A GENERAL TOOL

Any swarm, and deeper forensics

- Reads JSONL, JSON, CSV, plain logs, folders, and OpenTelemetry GenAI, LangSmith, Langfuse, AutoGen, CrewAI, OpenAI Agents SDK and MCP traces.
- Agents tab: which tokens are one agent, with evidence and confidence.
- Live mode and editable alert rules; case files; labels; compare two swarms.
- AI investigator (any OpenAI-compatible or local model) whose answers are charts with every id checked.
- **Forensics tab:** trace any string; credential lifecycles; command and control; dead drops; program evolution with diffs; attack stage ladder; lead–lag; anomalies; a story from the data; what-if; chain of custody; STIX and Timesketch exports.

Tested on two very different swarms: code payloads and wiki edits

TRY IT

pipx install git+https://github.com/itsfingerlickinggood/swarm-tracer@v1.0.0

then `swarm-tracer demo`, or `swarm-tracer serve --input your-swarm.jsonl`

Code: github.com/itsfingerlickinggood/swarm-tracer
Website, docs and video: itsfingerlickinggood.github.io/swarm-tracer